YouTube Comments Sentiment Analysis System

Using RoBERTa Transformer Model with Interactive Streamlit Dashboard

Authors

#	Name	Registration No.
1	Ali Hassan	2023-BS-AI-038
2	Hanzla Bhatti	2023-BS-AI-046
3	Waleed Amjad	2023-BS-AI-054
4	Taha Rehman	2023-BS-AI-090

Evaluator: Mr. Muhammad Arham | Department of Artificial Intelligence, The University of Faisalabad

Repository: github.com/waleed-Amjad/Youtube-Sentiment-Analysis | May 2026

Abstract

This article presents a comprehensive overview of the YouTube Comments Sentiment Analysis System (YT-Sentiment), an end-to-end web-based application developed as part of the Bachelor of Science in Artificial Intelligence program at the University of Faisalabad. The system leverages the RoBERTa transformer model (cardiffnlp/twitter-roberta-base-sentiment-latest) from Hugging Face to perform three-class sentiment classification — Positive, Negative, and Neutral — on large-scale real-world YouTube comment datasets. The platform integrates an automated NLTK preprocessing pipeline, GPU-accelerated PyTorch inference, a Streamlit interactive dashboard with KPI metrics, keyword frequency analytics, word cloud visualizations, and a live prediction interface. This article details the system scope, functional and non-functional requirements, technology stack, architecture, and constraints, providing a holistic technical reference for developers, evaluators, and researchers.

Keywords: *Sentiment Analysis, RoBERTa, Transformer, NLP, YouTube Comments, Streamlit, Deep Learning, GPU Inference, CUDA, Text Classification*

1. Introduction

The proliferation of user-generated content on platforms such as YouTube has created an unprecedented need for automated tools capable of extracting sentiment and opinion from large comment corpora. Manual annotation of millions of comments is infeasible, making deep learning-based sentiment analysis a critical technique for content creators, researchers, and platform moderators.

The YouTube Comments Sentiment Analysis System — YT-Sentiment — addresses this challenge by combining state-of-the-art transformer-based NLP with an accessible web interface. Built upon the RoBERTa architecture, which represents a robustly optimized variant of BERT with superior pretraining strategies, the system classifies comments into Positive, Negative, and Neutral categories with high accuracy.

This article documents the system in its entirety, covering its purpose, scope, functional design, non-functional requirements, architecture, and known constraints, drawing from the formal Software Requirements Specification (SRS) prepared for academic evaluation.

2. System Overview and Scope

YT-Sentiment is designed as a self-contained, standalone web application that does not depend on external enterprise systems. It interfaces with four primary external components: the Hugging Face Transformers Hub as the source of the pretrained RoBERTa model; the YouTube Comments Sentiment Dataset hosted on Kaggle as the labeled evaluation corpus; the PyTorch/CUDA runtime for GPU-accelerated inference; and NLTK for tokenization and stop-word removal during text preprocessing.

2.1 Core Product Features

The system delivers eight principal capabilities:

- Automated preprocessing pipeline incorporating tokenization, stop-word removal, URL/HTML stripping, and text normalization using NLTK.
- RoBERTa-based three-class sentiment classification producing Positive, Negative, and Neutral labels.
- Configurable sample size selection via a sidebar slider (2,000 to 40,000 comments in steps of 1,000).
- KPI dashboard displaying total analyzed comments, sentiment distribution percentages, model accuracy, precision, recall, and weighted F1-score.
- Per-class and overall keyword frequency bar charts for the top 20 and top 15 terms respectively.
- Interactive word cloud visualizations generated per sentiment class.
- Live prediction interface accepting user-supplied comment text for real-time classification.

- Dynamic GPU/CPU detection with automatic device allocation and fallback.

3. User Classes and Operating Environment

The system targets three distinct user classes. Academic evaluators, typically faculty members with moderate familiarity with machine learning concepts, use the dashboard to assess project quality and system functionality. Developer researchers — the project team itself — interact with and extend the codebase, requiring proficiency in Python, ML frameworks, and NLP pipelines. End users, who may have no coding background, rely exclusively on the Streamlit web interface to explore sentiment trends.

The operating environment mandates Windows 10/11 or Linux Ubuntu 20.04+, a minimum of 8 GB RAM, and ideally an NVIDIA GPU with at least 4 GB VRAM (CUDA 11.8 compatible) for accelerated inference, though CPU fallback is fully supported. Python 3.9 or higher is required, and the dashboard is accessible from any modern web browser.

4. Functional Requirements

4.1 Data Loading and Management

On application startup, the system loads the YouTube comments dataset from data/youtube_comments_cleaned.csv. The file must contain at minimum a CommentText column and a Sentiment column with values restricted to Positive, Negative, or Neutral. Datasets and models are cached using Streamlit's @st.cache_data and @st.cache_resource decorators, preventing redundant I/O and model reloading across user interactions and ensuring a responsive interface.

4.2 Text Preprocessing Pipeline

All comments pass through an automated cleaning pipeline prior to analysis. The pipeline applies lowercase conversion, removal of URLs and HTML tags, elimination of special characters and punctuation, NLTK-based tokenization, and English stop-word removal. Processing time (in seconds) and the active device name (GPU or CPU) are measured and surfaced to the user as part of the pipeline output summary.

4.3 Sentiment Classification

The system loads the pretrained RoBERTa model from the Hugging Face Transformers library, assigning it to a CUDA-compatible GPU where available or falling back to CPU otherwise. Batch inference runs with a configurable batch size of 8, with each comment truncated to a maximum token length of 512. The model returns probability scores across all three sentiment classes, which are then mapped to standardized labels (Positive, Negative, Neutral) regardless of case. Any unmapped label defaults to Neutral.

A sidebar checkbox labeled 'Run BERT Model Evaluation' toggles evaluation mode. When active, the model evaluates up to 1,000 comments from the sampled dataset. When disabled, the system uses a fixed 8,000-comment sample for KPI computation only, bypassing model inference entirely.

4.4 KPI Dashboard

The dashboard header displays four metrics in a columnar layout: total comments analyzed (integer with comma formatting), positive sentiment percentage (one decimal place), negative sentiment percentage, and neutral sentiment percentage. When evaluation is enabled, scikit-learn is used to compute and display overall accuracy, weighted precision, weighted recall, weighted F1-score, and a per-class classification report in tabular format showing precision, recall, F1-score, and support for each class.

4.5 Keyword Analysis and Visualization

Three layers of keyword analysis are provided. The first computes and charts the top 20 most frequent keywords across all comments using a horizontal bar chart. The second computes and charts the top 15 most frequent keywords per sentiment class, with each class rendered in a separate tabbed bar chart. The third generates word cloud images for each class at 800×400 pixels, white background, with a maximum of 150 words. All three word clouds are displayed side-by-side in a three-column layout.

4.6 Live Prediction Interface

Users may paste one or more YouTube comments (one per line) into a text area of 150px display height. On clicking 'Analyze with BERT', the system splits the input by line, passes non-empty comments to the RoBERTa classifier, and returns the predicted sentiment label with confidence score and a color-coded emoji indicator: green circle for Positive, red circle for Negative, and white circle for Neutral. A loading spinner displaying the active device name provides feedback during inference.

5. Non-Functional Requirements

The non-functional requirements govern the system's performance, reliability, usability, security, maintainability, and portability characteristics.

5.1 Performance

On GPU hardware (NVIDIA GTX 1650, 4 GB VRAM), the system targets a throughput of at least 50 comments per second during inference. CPU fallback must sustain at least 5 comments per second. The initial dashboard load time with model caching must not exceed 10 seconds. Live prediction latency for up to 10 user-supplied comments on GPU must remain below 3 seconds. Full preprocessing of 40,000 comments must complete within 60 seconds.

5.2 Reliability

The system must maintain stable operation during continuous sessions of up to four hours without memory leaks or crashes. GPU unavailability must trigger automatic CPU fallback without user intervention. Missing or malformed dataset rows must be silently skipped rather than causing application termination.

5.3 Usability

The dashboard must be navigable by a non-technical user within five minutes of first use without training or documentation. All KPI metrics must be clearly labeled with units and descriptions. Sidebar settings must be logically organized with descriptive headers. Sentiment color-coding must adhere to intuitive conventions: green for positive, red for negative, and white/grey for neutral.

5.4 Security

The application does not handle personally identifiable information and therefore requires no user authentication. The dataset is stored locally and never transmitted to external servers. All model inference is performed on the local machine; no comment data is sent to third-party APIs during prediction.

5.5 Maintainability and Portability

The codebase is modularized into app.py for UI orchestration and utils.py for preprocessing and keyword utilities. All functions follow Python PEP 8 conventions with descriptive naming. Dependency versions are pinned in requirements.txt. The application must run on any system supporting Python 3.9+ and pip, be deployable to Streamlit Cloud without code modification, and use only relative file paths with no platform-specific hard-coding.

6. System Architecture

6.1 Three-Layer Architecture

YT-Sentiment follows a three-layer architectural pattern. The Data Layer encompasses CSV dataset storage, the NLTK preprocessing pipeline, and Streamlit-based data caching. The Model Layer hosts the Hugging Face Transformers pipeline with the RoBERTa model and the PyTorch inference engine with CUDA support. The Presentation Layer is the Streamlit web application, delivering the interactive dashboard, KPIs, visualizations, and live prediction panel.

6.2 Technology Stack

The technology stack reflects a carefully chosen set of best-in-class tools for each system layer.

Layer	Technology	Purpose
Frontend/UI	Streamlit	Interactive web dashboard
Model Inference	HuggingFace + PyTorch	RoBERTa sentiment classification
Preprocessing	NLTK, pandas	Text cleaning & dataset management
Visualization	Matplotlib, Seaborn, WordCloud	Charts and word clouds
Evaluation	scikit-learn	Accuracy, Precision, Recall, F1
GPU Acceleration	PyTorch CUDA 11.8	Accelerated inference

6.3 Data Flow

The primary data flow proceeds as follows. The user launches the Streamlit application. The application loads and caches the CSV dataset, then applies text preprocessing via utils.clean_text(). The RoBERTa model is loaded from Hugging Face (cached after the initial download). In evaluation mode, the model performs batch inference on up to 1,000 comments. KPI metrics are computed and rendered in the dashboard header. Keyword frequencies are computed per class and rendered as bar charts. Word clouds are generated and displayed in the visualization tab. The user may optionally invoke the live prediction panel to classify custom comment text.

6.4 Module Descriptions

The app.py module serves as the application entry point and orchestrator. Its responsibilities include GPU/CPU device detection, model loading with caching, dataset loading and sampling, preprocessing invocation, KPI computation and rendering, visualization rendering, and live prediction interface management.

The utils.py module provides reusable helper functions. clean_text() performs text normalization and stop-word removal. get_top_keywords() computes overall term frequencies across all comments. get_top_keywords_per_sentiment() performs the same computation segmented by sentiment class.

7. Constraints and Limitations

7.1 Hardware Constraints

GPU VRAM is capped at 4 GB, which constrains the inference batch size to 8 and limits the evaluation subset to 1,000 comments per run. CPU inference is approximately ten times slower than GPU operation; large-sample evaluations are therefore not recommended in CPU-only environments.

7.2 Software Constraints

The system is constrained to the cardiffnlp/twitter-roberta-base-sentiment-latest model; integrating an alternative model would require code modifications. The dataset must contain CommentText and Sentiment columns with exact name matching. Streamlit's execution model re-runs the entire script on every user interaction, a limitation mitigated by caching but which still constrains stateful processing.

7.3 Academic Scope Limitations

As an academic proof-of-concept, YT-Sentiment does not include user authentication, persistent storage, or enterprise-grade scalability. Sentiment labels in the dataset are accepted as ground truth without re-verification or human annotation

review. The system supports English-language text only and does not extend to multilingual comment analysis.

8. Requirements Traceability

The table below maps each functional requirement identifier to its implementing module and assigned priority.

Req. ID	Description	Module	Priority
FR-1.1	Dataset Loading	app.py	High
FR-1.2	Sample Configuration	app.py	Medium
FR-1.3	Data Caching	app.py	High
FR-2.1	Text Cleaning	utils.py	High
FR-2.2	Processing Time Tracking	app.py	Low
FR-3.1	Model Loading	app.py	High
FR-3.2	Batch Inference	app.py	High
FR-3.3	Label Mapping	app.py	High
FR-3.4	Evaluation Toggle	app.py	Medium
FR-4.1	Summary KPIs	app.py	High
FR-4.2	Model Performance Metrics	app.py	High
FR-5.1	Overall Keywords	utils.py, app.py	Medium
FR-5.2	Per-Sentiment Keywords	utils.py, app.py	Medium
FR-5.3	Word Clouds	app.py	Medium
FR-6.1	User Input Interface	app.py	High
FR-6.2	Real-Time Classification	app.py	High
FR-6.3	Spinner Feedback	app.py	Low

9. Conclusion

The YouTube Comments Sentiment Analysis System represents a complete, production-quality academic implementation integrating modern NLP, deep learning inference, and interactive web-based visualization. By deploying the RoBERTa transformer model within a Streamlit interface with GPU acceleration, the system achieves a practical and scalable solution for large-scale sentiment mining of social media comment data.

The modular two-file architecture, strict adherence to PEP 8, pinned dependencies, and Streamlit Cloud portability collectively ensure maintainability and reproducibility. Future extensions could include multilingual support, integration with the YouTube Data API for live comment ingestion, user authentication for enterprise deployment, and persistent storage for longitudinal trend analysis.

References

[1] Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.

[2] Liu, Y. et al. (2019). RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692.

[3] Barbieri, F. et al. (2022). TweetEval: Unified Benchmark and Comparative Evaluation for Tweet Classification. HuggingFace Model Hub.

[4] Streamlit Documentation. https://docs.streamlit.io

[5] YouTube Comments Sentiment Dataset by Amaan Poonawala. Kaggle.

[6] IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.